



Amazon Simple Queue Service (SQS)

Lecture Slides

These lecture slides are provided for personal and non-commercial use only

Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

Introduction to SQS

What is a Messaging System?

Used to provide asynchronous communication and decouple processes via messages / events
From a sender and receiver (producer and consumer)

What is a Queueing System?

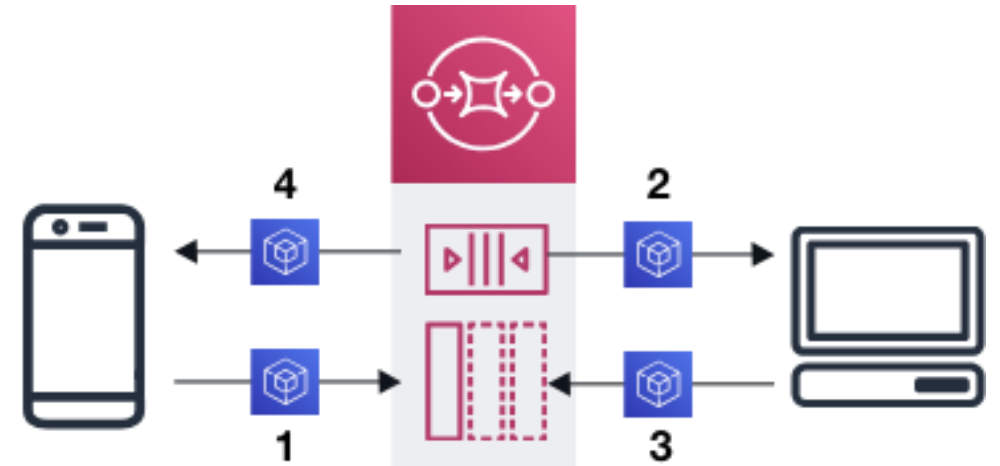
A Queueing system is a messaging system that generally will delete messages once they are consumed. Simple communication. **Not Real-time**. Have to pull. Not reactive.



Simple Queueing Service (SQS)

Fully managed **queuing service** that enables you to decouple and scale microservices, distributed systems, and serverless applications

Use Case: You need to queue up transaction emails to be sent e.g. Signup, Reset Password.



Introduction to SQS

SQS Queue types:

Standard — allows you to process a nearly unlimited number of messages, but messages are out of order

FIFO — allows you to process a certain amount of message, but guarantee order of messages and no duplicate messages

Message Size

The size of a message can be between **1 byte and 256 KiB**

For larger messages you need to use a library.

Message Retention is how long SQS will hold onto a message in the queue before dropping it from the queue (deleting it)

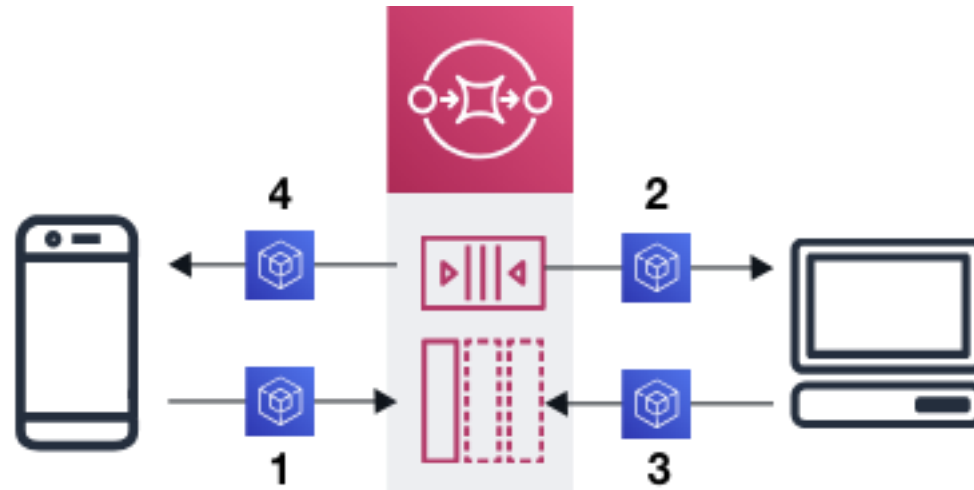
- Message retention by **default is 4 days**.
- Message retention can be adjusted from a minimum of **60 seconds** to a **max of 14 Days**

Queue Encryption

SQS queue can be encrypted using Amazon managed Server-Side-Encryption (SSE) or using KMS

SQS - Use Case

1. App publishes messages to the queue
2. Another app pulls the queue and finds the message, and does something
3. Another app reports that they completed their task and marks the message for completion
4. Original app pulls the queue and sees the message is no longer in the queue.



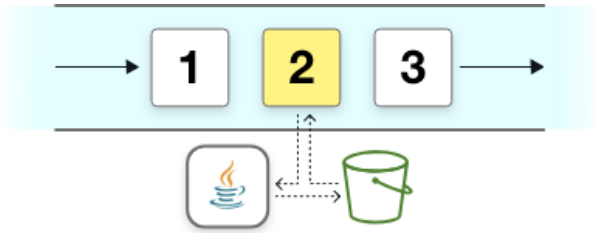
Both apps are using the **AWS SDK** to push messages and pull the queue.

SQS – Sending Large Messages

To send large messages to SQS you can use the the libraries:

- Amazon SQS Extended Client Library for Java/Python

<https://github.com/aws-labs/amazon-sqs-python-extended-client-lib>



These libraries are useful for messages that are larger than the current maximum of 256 KB, **with a maximum of 2 GB**.

Both libraries save the actual payload to an **S3 bucket**, and publish the reference of the stored S3 object to the SNS topic.

```
import boto3
import sns_extended_client

sns = boto3.client('sns')
sns.large_payload_support = 'bucket-name'

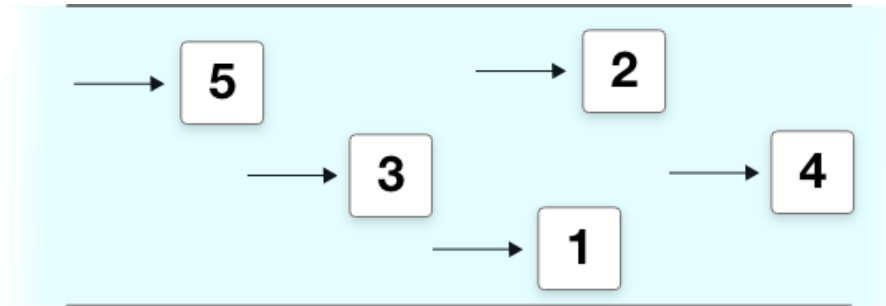
# boto SNS.Topic resource
resource = boto3.resource('sns')
topic = resource.Topic('topic-arn')
platform_endpoint = resource.PlatformEndpoint('endpoint-arn')
platform_endpoint.large_payload_support = 'my-bucket-name'

# If you want to keep it always enabled.
platform_endpoint.always_through_s3 = True

# publish the large message
sns.publish(
    Message="message",
    MessageAttributes={
        "S3Key": {
            "DataType": "String",
            "StringValue": "---S3--Key--",
        }
    },
)
```

SQS - Standard Queue

AWS SQS Standard queue allows you allow you a nearly-unlimited number of transactions per second (tps).



- AWS SQS Standard Queues Guarantees that a message will be delivered AT LEAST once.
- More than one copy of a message could be potentially delivered out of order.
 - You need to ensure your consumers can process messages that arrive more than once and out of order
- Provides best-effort ordering that helps ensure a message is generally delivered in the same order that it was sent

SQS – Standard Queue

Sending a message to the SQS
Queue using the AWS SDK

```
require 'aws-sdk-sqs'

sqs_client = Aws::SQS::Client.new(region: 'us-west-2')

queue_url = 'https://sqs.us-west-2.amazonaws.com/123456789012/my-queue'

message_body = 'Hello, World!'

begin
  sqs_client.send_message(queue_url: queue_url, message_body: message_body)
  puts "Message sent successfully."
rescue Aws::SQS::Errors::ServiceError => e
  puts "Error sending message: #{e}"
end
```

Example using the ruby SDK.

SQS – Standard Messages

Sending batch of messages to the
SQS Queue using the AWS SDK

```
require 'aws-sdk-sqs'

sqs_client = Aws::SQS::Client.new(region: 'us-west-2')

queue_url = 'https://sqs.us-west-2.amazonaws.com/123456789012/my-queue'

messages = [
  {id: 'message1', message_body: 'First Message'},
  {id: 'message2', message_body: 'Second Message'},
  {id: 'message3', message_body: 'Third Message'}
]

begin
  sqs_client.send_message_batch({
    queue_url: queue_url,
    entries: messages.map { |msg| {id: msg[:id], message_body:
msg[:message_body]} }
  })
  puts "Messages sent in batch successfully."
rescue Aws::SQS::Errors::ServiceError => e
  puts "Error sending messages in batch: #{e}"
end
```

Example using the ruby SDK.

SQS – FIFO Queue

AWS SQS First-In-First-Out (FIFO) guarantees the order of messages when being consumed



- Limited to 300 transactions per second (TPS)
- Messages have a unique deduplication ID to ensure there are no duplicate messages in the queue
 - Since there are no duplicates FIFO ensures **Exactly-once processing**
- Messages are ordered based on Message Group Id.
- To ensure order is preserved each Producer should use their own unique Message Group Id.
- To request (poll) messages your Consumer has to specify a Message Group Id
- Its possible to read upto 10 messages a time.
- SQS FIFO manages data in partitions across multiple AZs, but its all managed for you by AWS.
 - Each partition supports 3,000 messages per second with batching
 - Or up to 300 messages per second for send, receive, and delete operations
- You can't convert an existing standard queue into a FIFO queue.

SQS - FIFO Queue

Sending message to a FIFO queue.

Note that you have to provide:

- message_group_id
- Message_deduplication_id

```
require 'aws-sdk-sqs'

sqs_client = Aws::SQS::Client.new(region: 'us-west-2')

queue_url = 'https://sqs.us-west-2.amazonaws.com/123456789012/my-queue.fifo'

message_body = 'Hello, FIFO World!'
message_group_id = 'myMessageGroup1'
message_deduplication_id = 'myMessageDeduplicationId1'

begin
  sqs_client.send_message(
    queue_url: queue_url,
    message_body: message_body,
    message_group_id: message_group_id,
    message_deduplication_id: message_deduplication_id
  )
  puts "Message sent to FIFO queue successfully."
rescue Aws::SQS::Errors::ServiceError => e
  puts "Error sending message to FIFO queue: #{e}"
end
```

SQS – FIFO Queue

High Throughput can be enabled on SQS FIFO queue to allow for 3,000 messages per second with batching (10x greater than SQS FIFO queue without High Throughput)

To enable High Throughput you set **FifoThroughputLimit=perMessageGroupId**

```
aws sqs set-queue-attributes \  
--queue-url "https://sqs.ca-central-1.amazonaws.com/123456789012/my-queue.fifo" \  
--attributes \  
FifoQueue=true,\  
ContentBasedDeduplication=true,\  
DeduplicationScope="messageGroup"\  
FifoThroughputLimit="perMessageGroupId"
```



SQS - ABAC

Attribute-based access control (ABAC) is an authorization process that defines permissions based on tags that are attached to users and AWS resources.

SQS supports ABAC by allowing you to control access to your Amazon SQS queues based on the tags and aliases that are associated with an Amazon SQS queue.

Possible condition tags:

- aws:ResourceTag
- aws:RequestTag
- aws:TagKeys

Example of **denying production resources** (tagged with prod) from sending, receiving or delete messages to a queue



```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyAccessForProd",
      "Effect": "Deny",
      "Action": "sqs:*",
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "aws:ResourceTag/environment": "prod"
        }
      }
    }
  ]
}
```

SQS – Access Policy

Access Policy allows you grant to grant other principals permission to the SQS Queue.

Common actions:

- SendMessage
- ReceiveMessage
- Delete Message

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowSNSPublish",
      "Effect": "Allow",
      "Principal": {
        "Service": "sns.amazonaws.com"
      },
      "Action": "sqs:SendMessage",
      "Resource": "arn:aws:sqs:region:account-id:queue-name",
      "Condition": {
        "ArnEquals": {
          "aws:SourceArn": "arn:aws:sns:region:account-id:topic-name"
        }
      }
    }
  ]
}
```

Example of letting an SNS Topic send messages to the queue

SQS – Message Metadata

Message Metadata allows you to attach **metadata to your SQS messages**

Possible logical data types

- String
- Number
- Binary
- Custom
 - Just append a custom-type label to any data type eg.
 - Number.byte, Number.short
 - Binary.gif, Binary.png

```
aws sqs send-message \  
--queue-url "https://sqs.region.amazonaws.com/123456789012/my-queue" \  
--message-body "Your message text" \  
--message-attributes file://message-attributes.json
```



```
{  
  "OrderID": {  
    "DataType": "String",  
    "StringValue": "123456789"  
  },  
  "CustomerEmail": {  
    "DataType": "String",  
    "StringValue": "customer@example.com"  
  },  
  "PurchaseDate": {  
    "DataType": "String",  
    "StringValue": "2024-03-16"  
  },  
  "IsPriority": {  
    "DataType": "String",  
    "StringValue": "true"  
  },  
  "OrderTotal": {  
    "DataType": "Number",  
    "StringValue": "99.99"  
  },  
  "OrderItems": {  
    "DataType": "Binary",  
    "BinaryValue": "T3JkZXJJdGVtc0JpbmFyeURhdGE="  
  }  
}
```

SQS – Visibility Timeout

Visibility timeout is a period of time message will be invisible after they are read/consumed by an application to avoid being read and processed by other applications

Default: 30 second

Min : 0 seconds

Max: 43200 seconds (12 hours)

Set **VisibilityTimeout** via the AWS CLI



```
aws sqs set-queue-attributes \  
--queue-url "https://sqs.region.amazonaws.com/123456789012/my-queue" \  
--attributes '{"VisibilityTimeout":"120"}'
```

a message is hidden only after it is consumed from the queue.

SQS – Delay Queues

Delay queues let you **postpone the delivery of new messages to consumers** for a number of second when your app needs more time



Any messages that you send to a delay queue remain invisible to consumers for the duration of the delay period.

- Default Value: 0 seconds
- Max Value: 900 seconds (15 minutes)
- **Standard queue:** per-queue delay setting will only apply the new messages in the queue
- **FIFO queue:** per-queue delay setting will apply to all messages in the queue

Set **DelaySeconds** via the AWS CLI

```
aws sqs set-queue-attributes \  
--queue-url "https://sqs.region.amazonaws.com/123456789012/my-queue" \  
--attributes '{"DelaySeconds":"60"}'
```

a message is hidden when it is first added to queue

SQS – Message Timers

Message Timers let you specify an initial invisibility period **for an individual message** when sending to the queue

Set **DelaySeconds** via the AWS CLI

```
aws sqs send-message \  
--queue-url "https://sqs.region.amazonaws.com/123456789012/my-queue" \  
--message-body "Your message text" \  
--delay-seconds 10
```



Both Delay Queue and Message Timers use the property called DelaySeconds

FIFO queues don't support timers on individual messages.

SQS – Temporary Queues

Temporary queues are high-throughput, cost-effective, application-managed temporary queues when using common message patterns such as request-response

Temporary Queue Client (written in Java) lets you create lightweight queues that are deleted automatically when they are no longer in use.

<https://github.com/aws-labs/amazon-sqs-java-temporary-queues-client>

benefits of temporary queues:

- serve as lightweight communication channels for specific threads or processes.
- can be created and deleted without incurring additional cost.
- are API-compatible with static (normal) Amazon SQS queues.

```
AmazonSQS sqs = AmazonSQSClientBuilder.defaultClient();
AmazonSQSResponder responder = AmazonSQSResponderClientBuilder.standard()
    .withAmazonSQS(sqs)
    .build();

SQSMessageConsumer consumer = SQSMessageConsumerBuilder.standard()
    .withAmazonSQS(responder.getAmazonSQS())
    .withQueueUrl(queueUrl)
    .withConsumer(message -> {
        int x = ThreadLocalRandom.current().nextInt(10) + 1;
        String responseBody = "Here are " + x + " more widgets. Enjoy!";
        System.out.println("Sending reply: " + responseBody);
        responder.sendResponseMessage(MessageContent.fromMessage(message),
            new MessageContent(responseBody));
    }).build();
consumer.start();
```

SQS - Short vs Long Polling

Polling is the method by which we retrieve messages from the queues.

Short Polling (default)

returns messages immediately, even if the message queue being polled is empty.

When you need a message **right away**

Long Polling

waits until message arrives in queue or till long poll timeout expires.

When you need to **save money** by reducing how often you poll

WaitTimeSeconds determines polling:

- Short Polling: 0 seconds
- Long Polling: > 0 seconds
- Max long polling is 20 seconds.



```
response = sqs_client.receive_message({  
  queue_url: queue_url,  
  max_number_of_messages: 10,  
  wait_time_seconds: 20 # Enables long polling for up to 20 seconds  
})
```

Example of long polling using AWS Ruby SDK v3